



ARA0110 PADRÕES DE PROJETO

Aula 13: Especificações Java - Parte 2



COMPLEXIDADE





COMPLEXIDADE

A especificação JEE trouxe uma grande revolução na construção de aplicações corporativas Java, o que proporcionou boa parte da popularidade que a plataforma possui. No entanto, usar JEE não é um trabalho simples, visto que há diversas complexidades que o desenvolvedor ainda precisa tocar quando se trabalha em um projeto que usa tal especificação.

COMPLEXIDADE

**Arquitetura
multicamadas**

**Excesso de
configuração**

**Ciclo de vida
de objetos**



SPRING

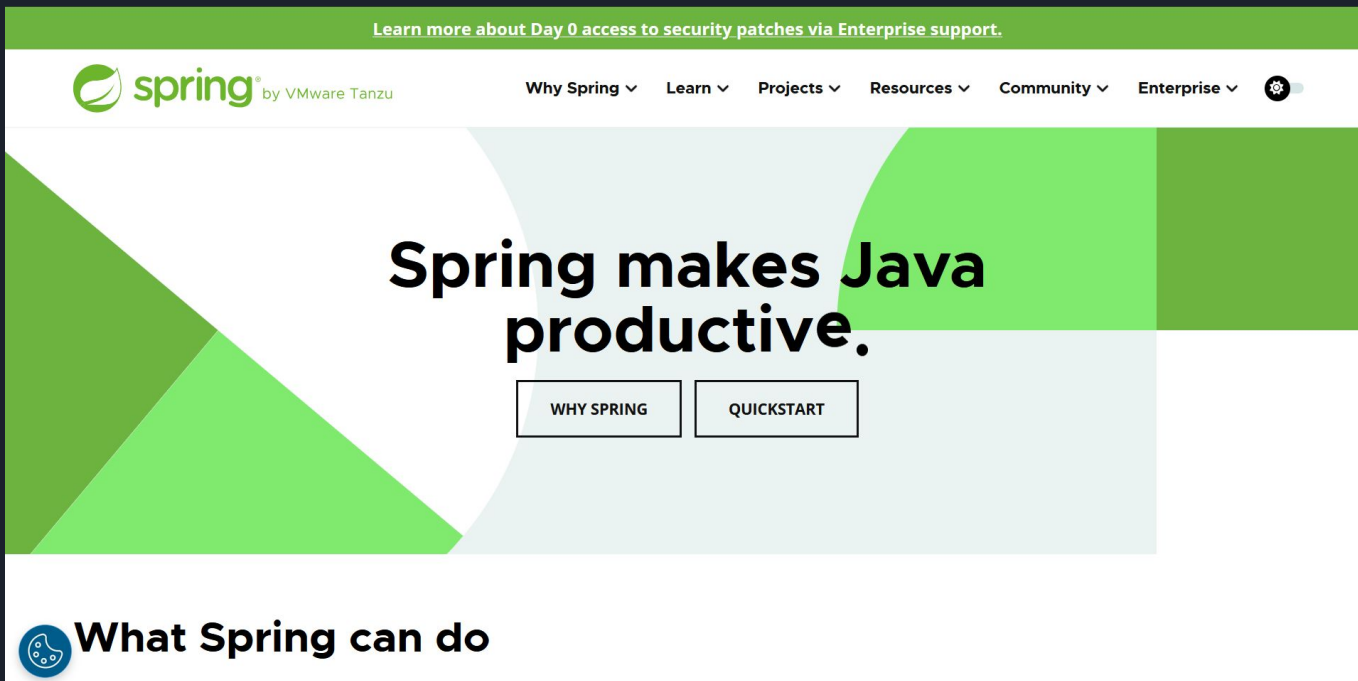
Nesse cenário, um novo projeto adentrou no mercado como uma alternativa direta e leve à extrema complexidade do JEE, focando na produtividade do desenvolvedor. Assim, no início dos anos 2000, foi lançado o **Spring Framework**, um projeto de framework de aplicações de código aberto com filosofias claras de infraestrutura moderna, contêineres de aplicação bem formados e abstrações de alto nível.

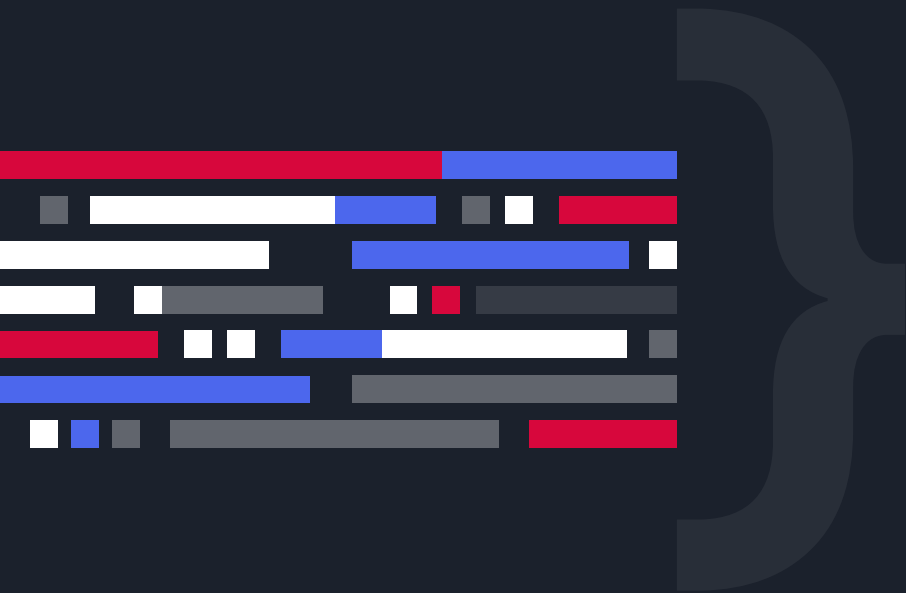
OBJETIVOS

- **Substituição dos EJBs Pesados:** Substituiu os complexos Enterprise JavaBeans (EJB) por objetos Java simples (POJO)
- **Inversão de Controle (IoC):** Introduziu a Injeção de Dependências para desacoplar o código e facilitar testes unitários
- **Redução de Código Boilerplate:** Eliminou os excessivos arquivos XML e códigos repetitivos de conexão com bancos de dados
- **Foco no Design Ágil:** Alinhou-se ao crescimento das metodologias ágeis, que exigiam entregas de software muito mais rápidas
- **Evolução para Microserviços:** Consolidou-se anos depois com o Spring Boot, ideal para arquitetura de microserviços em nuvem

PROJETO

<https://spring.io>





CRESCIMENTO

Com o passar do tempo, o projeto cresceu para atender diferentes necessidades de desenvolvimento moderno. Isso fez com que o Spring Framework começasse a assumir uma roupagem modularizada, onde cada projeto específico pode ser adicionado conforme a necessidade das aplicações.



PROJETOS

- **Spring Framework:** Núcleo do projeto com injeção de dependência, gestão de transações, aplicativos web e mais
 - **Spring Data:** Projeto de acesso consistente a bases de dados (relacionais e não relacionais)
 - **Spring Boot:** Facilita a criação de aplicações com servidores embutidos e configurações pré-definidas de ferramentas
 - **Spring Cloud:** Provê ferramentas de desenvolvimento para aplicações nativas em nuvem
 - **Spring Security:** Provê componentes de autenticação e autorização de aplicações
 - **Spring AI:** Provê integração com modelos de IA em aplicações Java
 - **Spring AMQP:** Integração com serviços de mensageria com o protocolo AMQP
- 



Spring
Boot

SPRING INITIALIZR

Para iniciar um novo projeto usando Spring Boot, podemos usar o site de configuração visual ofertado pelo projeto Spring, onde podemos configurar projetos e bibliotecas necessárias para a aplicação:

<https://start.spring.io>

SPRING INITIALIZR

start.spring.io

Estácio

Página inicial - Micros...

Hibernate with Gradle ...

regex101: build, test, a...

Download free 3D icon

WEB ACADEMY

Outros favoritos

 **spring**initializr

Project

☒ Gradle - Groovy

☐ Gradle - Kotlin

☐ Maven

Language

☒ Java

☐ Kotlin

☐ Groovy

Spring Boot

☐ 4.1.0 (SNAPSHOT)

☐ 4.1.0 (RC1)

☐ 4.0.7 (SNAPSHOT)

☒ 4.0.6

☐ 3.5.15 (SNAPSHOT)

☐ 3.5.14

Project Metadata

Group

com.example

Artifact

demo

Dependencies

ADD DEPENDENCIES... CTRL + B

No dependency selected



GENERATE CTRL + G

EXPLORE CTRL + SPACE

...

CRIAÇÃO DO PROJETO

Project

- ☐ Gradle - Groovy
- ☐ Gradle - Kotlin
- ☒ Maven

Language

- ☒ Java
- ☐ Kotlin
- ☐ Groovy

Spring Boot

- ☐ 4.1.0 (SNAPSHOT)
- ☐ 4.1.0 (RC1)
- ☐ 4.0.7 (SNAPSHOT)
- ☒ 4.0.6
- ☐ 3.5.15 (SNAPSHOT)
- ☐ 3.5.14

CRIAÇÃO DO PROJETO

Project Metadata

Group `br.estacio.amazonas`

Artifact `catalogo-produtos`

Package name `br.estacio.amazonas.catalogoprodutos`

Packaging ☒ Jar ☐ War

Configuration ☒ Properties ☐ YAML

Java ☐ 26 ☐ 25 ☒ 21 ☐ 17

CRIAÇÃO DO PROJETO

web

Press Ctrl for multiple adds

Spring Web **WEB**

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container. ↵

jpa

Press Ctrl for multiple adds

Spring Data JPA **SQL**

Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate. ↵

mysql

Press Ctrl for multiple adds

MySQL Driver **SQL**

MySQL JDBC driver. ↵

template

Press Ctrl for multiple adds

Thymeleaf **TEMPLATE ENGINES**

A modern server-side Java template engine for both web and standalone environments. Allows HTML to be correctly displayed in browsers and as static prototypes. ↵

CRIAÇÃO DO PROJETO

Dependencies

ADD DEPENDENCIES... CTRL + B

Spring Web **WEB**

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Spring Data JPA **SQL**

Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

MySQL Driver **SQL**

MySQL JDBC driver.

Thymeleaf **TEMPLATE ENGINES**

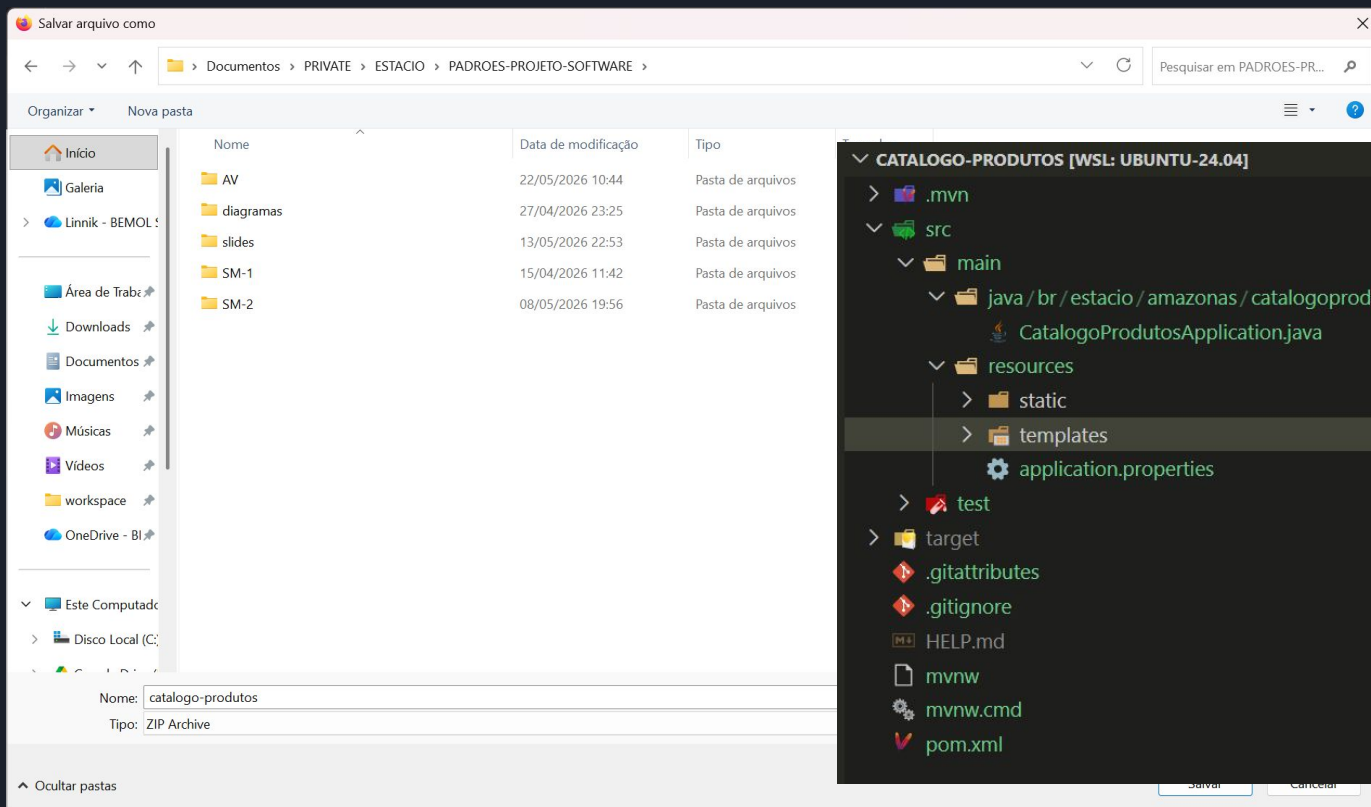
A modern server-side Java template engine for both web and standalone environments. Allows HTML to be correctly displayed in browsers and as static prototypes.



CRIAÇÃO DO PROJETO

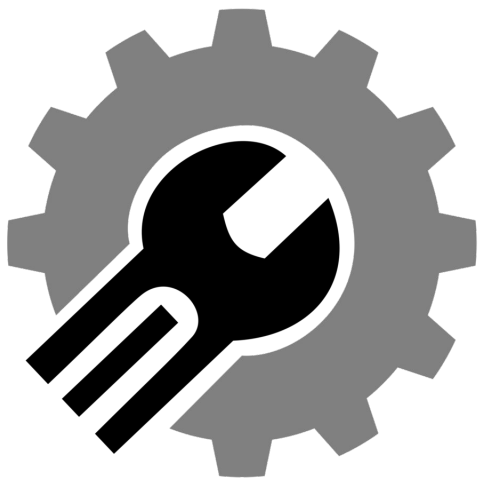
Após terminarmos a configuração do projeto, clicamos no botão "Generate". O Spring Initializr vai gerar um projeto pré-configurado com as dependências escolhidas, e vai retornar esse projeto em um pacote ZIP, que pode ser extraído e estará pronto para execução e início do desenvolvimento.

CRIAÇÃO DO PROJETO



COMPONENTES





CONFIGURAÇÃO

O Spring Framework trabalha com uma premissa de **Convenção sobre Configuração**. Isso significa que os componentes incorporados aos projetos já chegam previamente configurados com um cenário padrão de utilização. Quando há a necessidade de alguma configuração específica, isso pode ser feito através do arquivo "application.properties", que gerencia sua execução diretamente no container de aplicação Spring.

APPLICATION.PROPERTIES

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/catalogo_produtos
spring.datasource.username=admin
spring.datasource.password=admin@2026
spring.jpa.show-sql=true
spring.jpa.format-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
spring.jpa.properties.hibernate.hbm2ddl.auto=update
```



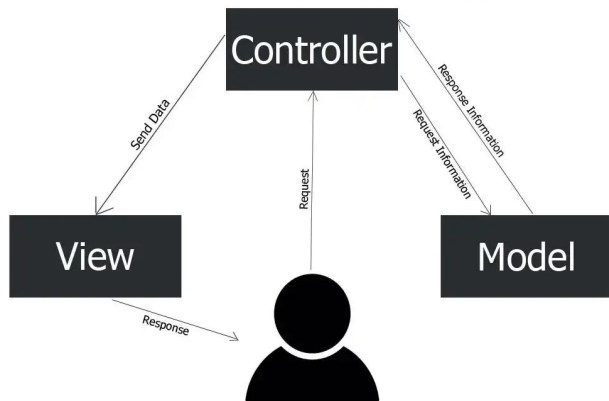
ANOTAÇÕES

Além do arquivo de propriedades, temos configurações que são feitas a partir de anotações disponibilizadas pelo ecossistema Spring. Podemos usar anotações para configurar controllers HTTP, componentes de tipos específicos (serviços, repositórios etc) entre outras configurações.

ANOTAÇÕES

```
package br.estacio.amazonas.catalogoprodutos;  
  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
  
@SpringBootApplication  
public class CatalogoProdutosApplication {  
  
    public static void main(String[] args) {  
        SpringApplication.run(CatalogoProdutosApplication.class, args);  
    }  
}
```

Model-View-Controller

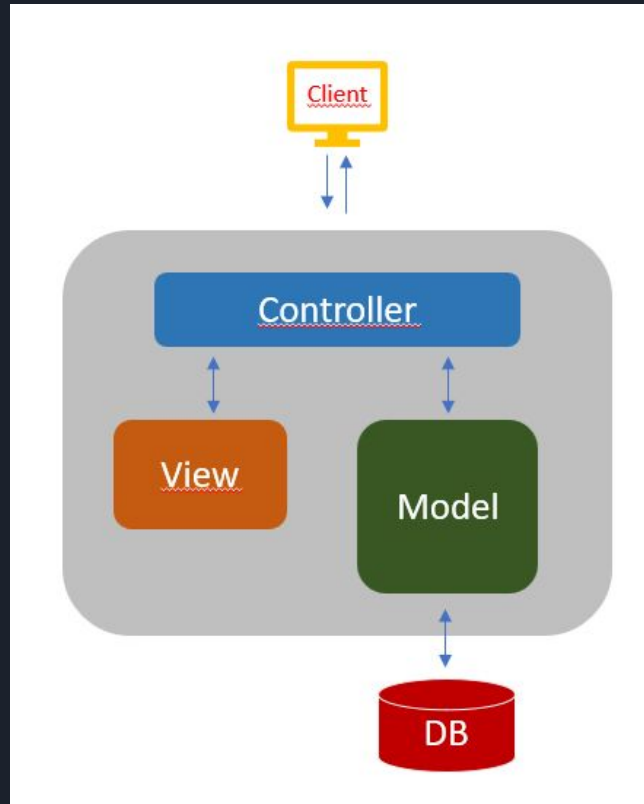


ARQUITETURA MVC

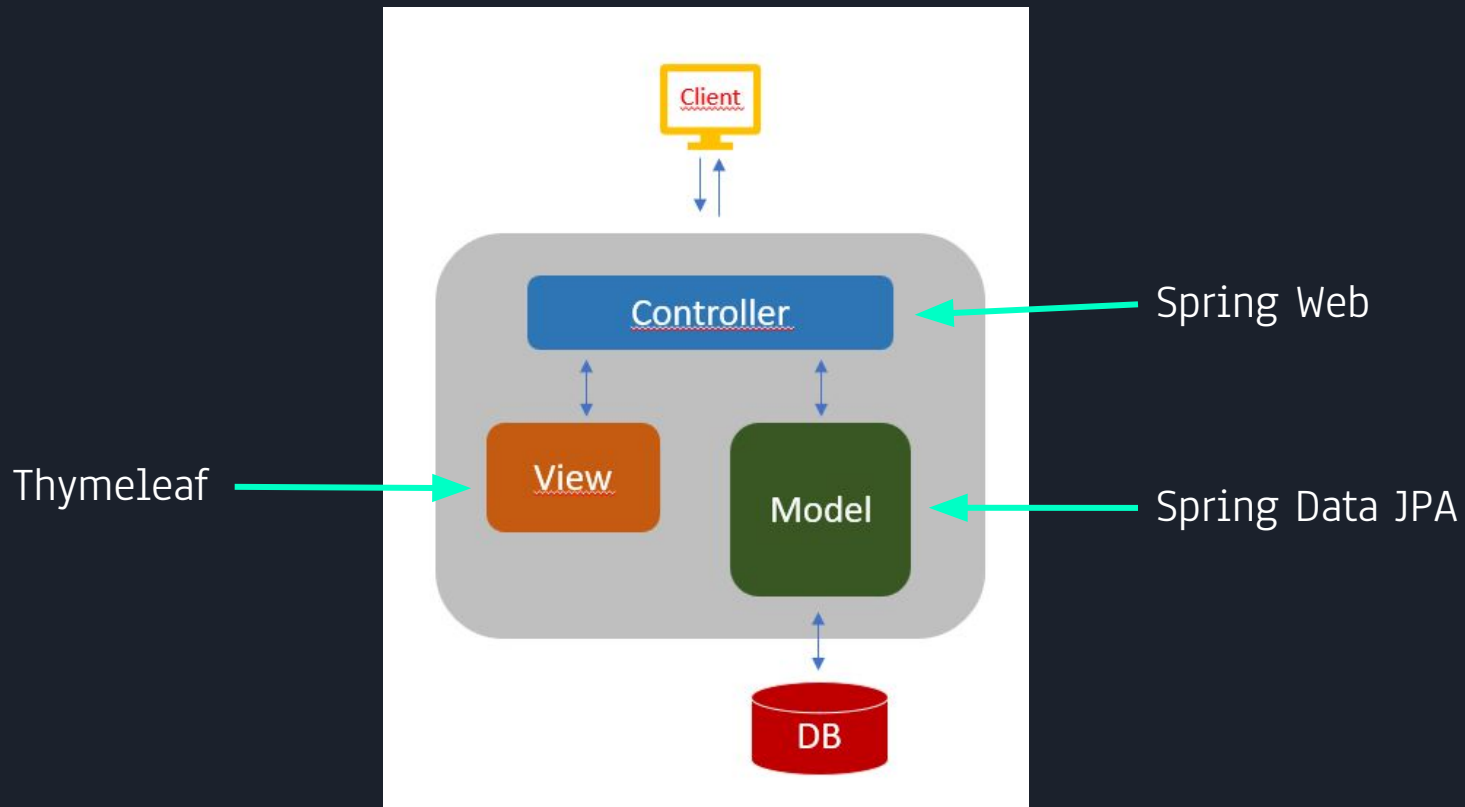
O padrão de arquitetura MVC prega que tenhamos camadas com responsabilidades específicas e dedicadas na orquestração da aplicação:

- **Modelo (M):** Camada que concentra regras de negócio e acesso a dados
- **Visão (V):** Camada de visualização e interação
- **Controlador (C):** Camada que coordena a interação entre modelo e visão

ARQUITETURA MVC



SPRING MVC



CONTROLLER

No Spring, definimos um controller a partir de uma classe que recebe a anotação **@Controller** (ou **@RestController**, para serviços web), e cada método deve ser capaz de atender uma requisição enviada em uma rota. Esses métodos serão atendidos mediante um método HTTP associado, tendo uma anotação para cada método.



HTTP

CONTROLLER

```
package br.estacio.amazonas.catalogoprodutos.controllers;  
  
import org.springframework.stereotype.Controller;  
import org.springframework.web.bind.annotation.GetMapping;  
  
@Controller  
public class HomeController {  
  
    @GetMapping(path = "/")  
    public String paginaHome() {  
        return "home";  
    }  
}
```

VIEW

Catálogo de Produtos [Produtos](#)

Spring Framework

Este é um sistema de exemplo para entendimento dos conceitos sobre o projeto Spring Framework. A aplicação em questão se trata de um catálogo de produtos.

© 2026 Catálogo de Produtos. Todos os direitos reservados.

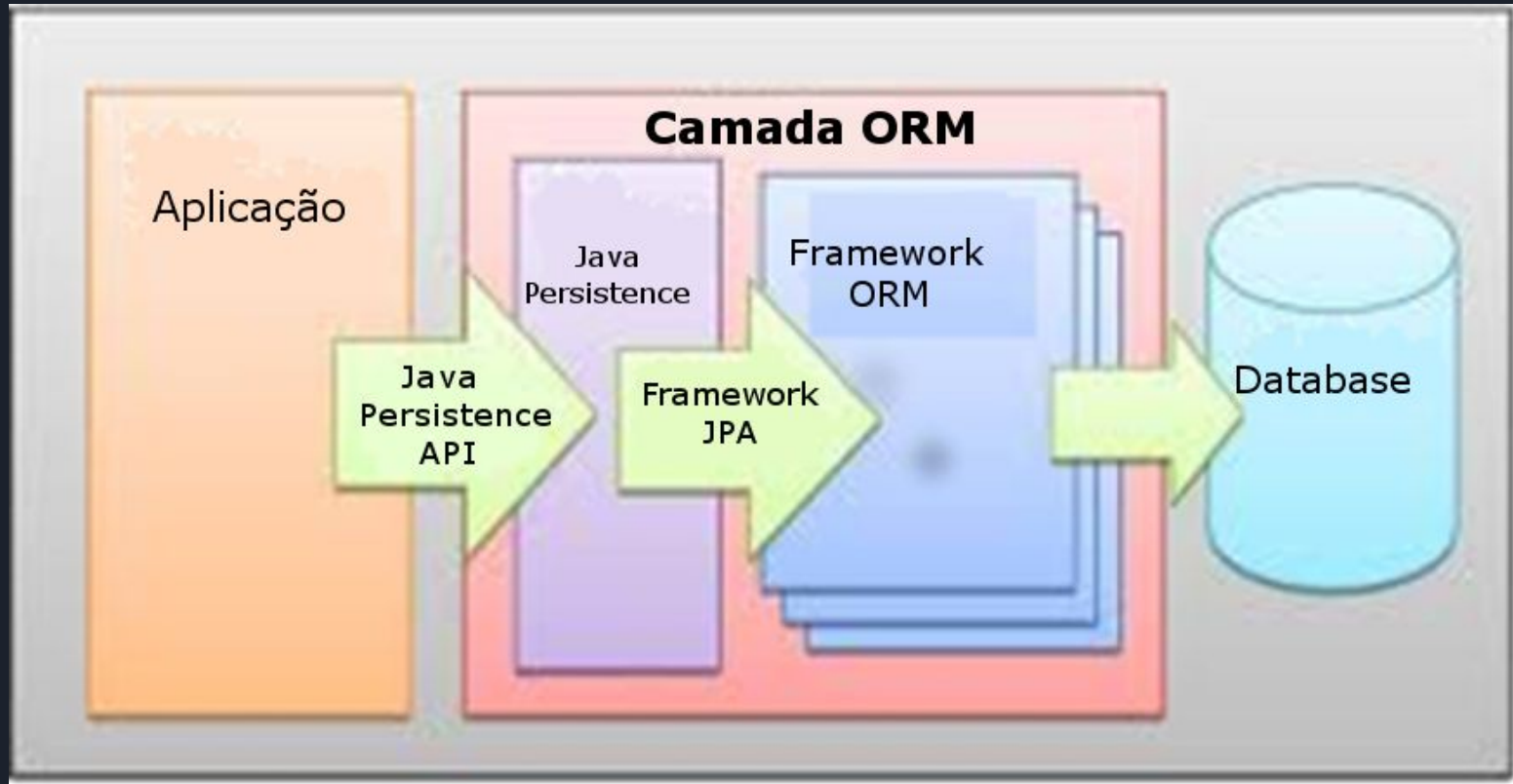


Spring Data JPA

SPRING DATA JPA

Sobre a camada da especificação JPA, a equipe do Spring construiu o projeto Spring Data JPA, que serve como uma camada de abstração focada em operações feitas no banco de dados, sem dependência explícita de usar uma instância de **EntityManager** ou mapear entidades em um arquivo "persistence.xml". Fora isso, toda a especificação JPA funciona exatamente da mesma forma em um projeto Spring.

JPA



MODELO

```
package br.estacio.amazonas.catalogoprodutos.models;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "produtos")
public class Produto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private Long id;

    @Column(name = "nome")
    private String nome;

    @Column(name = "preco")
    private Double preco;
}
```



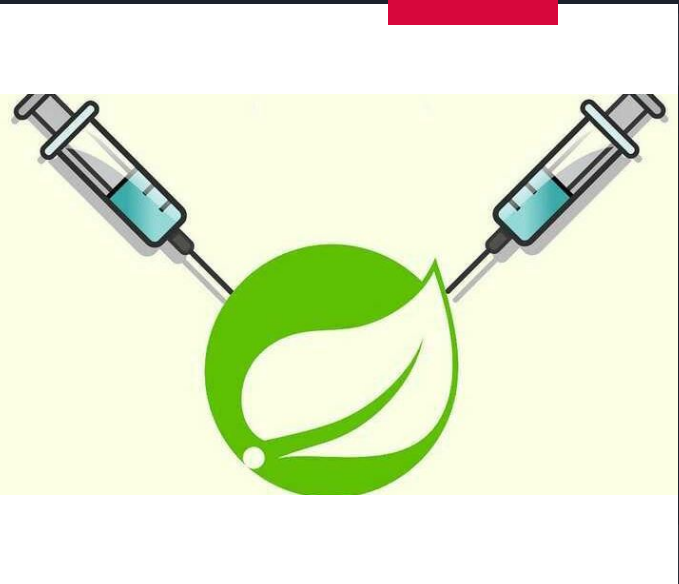
REPOSITÓRIOS

Os repositórios são elementos usados para abstrair o acesso a dados da camada de banco de dados para a camada de regras de negócio. O Spring possui abstrações para construção de repositórios com operações comuns como buscar, filtros específicos e persistência/remoção de registros em tabelas.

REPOSITÓRIOS

```
package br.estacio.amazonas.catalogoprodutos.repositories;  
  
import br.estacio.amazonas.catalogoprodutos.models.Produto;  
  
import org.springframework.data.jpa.repository.JpaRepository;  
import org.springframework.stereotype.Repository;  
  
@Repository  
public interface ProdutosRepository extends JpaRepository<Produto, Long> {}
```

Quando estendemos as interfaces do Spring Data JPA, ele fornece automaticamente uma implementação padrão para as operações de acesso a dados. Dessa forma, não há a necessidade de implementar uma classe para esta interface.



INJEÇÃO DE DEPENDÊNCIAS

Uma das principais vantagens de se trabalhar com o ecossistema Spring é a capacidade de usar injeção de dependência de forma facilitada através de anotações e reflexão. Todas as instâncias de objetos são gerenciados automaticamente pelo container de contexto de execução, e quando um objeto necessita de uma instância de componente específico, o container realiza a configuração adequada e o entrega a quem pede.

INJEÇÃO DIRETA

```
package br.estacio.amazonas.catalogoprodutos.services;

import br.estacio.amazonas.catalogoprodutos.repositories.ProdutosRepository;

import org.springframework.stereotype.Service;

@Service
public class ProdutosService {

    private ProdutosRepository repository;

    public ProdutosService(ProdutosRepository repository) {
        this.repository = repository;
    }
}
```

INJEÇÃO POR ANOTAÇÃO

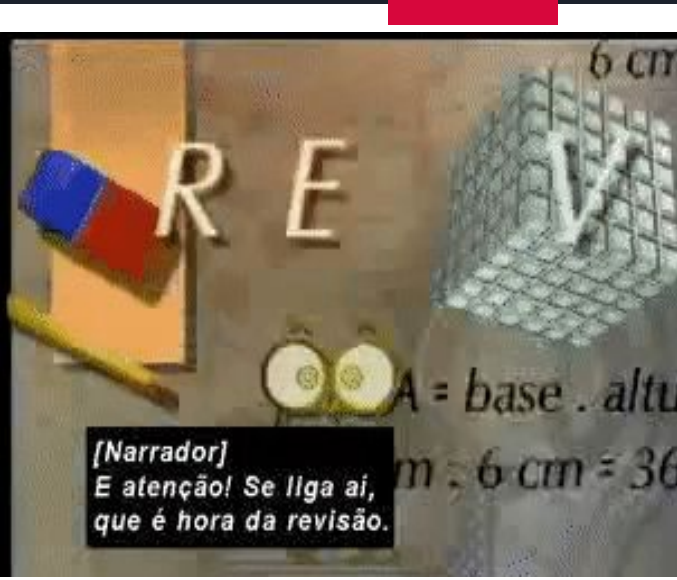
```
package br.estacio.amazonas.catalogoprodutos.controllers;

import br.estacio.amazonas.catalogoprodutos.services.ProdutosService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class HomeController {

    @Autowired
    private ProdutosService service;

    @GetMapping(path = "/")
    public String paginaHome() {
        return "home/index";
    }
}
```



RECAPITULANDO

- Complexidade da JEE
- Projeto Spring Framework
- Módulos Spring
- Spring Web MVC
- Spring Data JPA
- Inversão de Controle

OBRIGADO!



"TALVEZ NÃO FUNCIONE COM BURROS"
Shrek